

Introdução à Inteligência Artificial

1º ano - 1º semestre – 2025/26

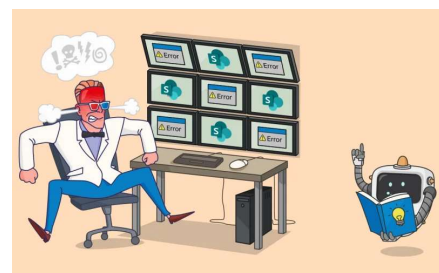
“3. Resolução de Problemas”

Professor Doutor **Ricardo Vardasca**
ricardo.vardasca@islasantarem.pt

1

Indice

- Resolução de problemas em IA
- Formulação de problemas
- Estruturação de problemas
- Problemas de procura
- Algoritmos de procura não informada
 - Procura em extensão (BFS)
 - Procura em profundidade (DFS)
 - Procura de custo uniforme (UFS)

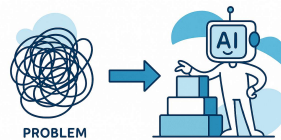


2

2

Resolução de problemas em IA

- Encontrar uma sequência de ações que leva de um **estado inicial** até a um **estado objetivo** (ou meta).
- Um agente de IA “resolve um problema” quando descobre como sair da situação atual para atingir um objetivo definido.



3

3

Formulação de problemas em IA

- Processo de definir claramente o problema que um agente inteligente deve resolver.
- Antes de o agente procurar uma solução, ele precisa entender **o que é o problema**, quais **ações pode realizar**, e qual é o **estado final desejado**.
- Um **modelo abstrato** transforma uma situação do mundo real num problema de procura que a IA pode processar.

4

4

Importância da formulação de problemas em IA

- Sem uma boa formulação:
 - O agente pode procurar soluções irrelevantes.
 - O espaço de procura pode tornar-se enorme e ineficiente.
 - Pode até resolver o problema errado!
- Exemplo:
 - Um carro autónomo precisa chegar ao destino.
 - Se a formulação não incluir estradas interditas ou trânsito, ele pode escolher um caminho “ótimo” que é impossível na realidade.

5

5

Estrutura de um problema em IA

Elemento	Descrição	Exemplo prático
Estado inicial	Situação inicial do agente (onde começa)	Localização atual de um robô
Ações possíveis	Conjunto de movimentos ou decisões que pode tomar	Mover para: norte, sul, leste, oeste
Função de transição	Como as ações mudam o estado atual	“Mover norte” → nova posição (x, y+1)
Objetivo (meta)	O estado desejado que o agente quer alcançar	Chegar ao destino ou resolver um puzzle

6

6

Componentes principais da formulação de um problema

Elemento	Significado	Exemplo prático (robô)
Estado inicial	Onde o agente começa.	Robô está na posição (0,0).
Ações possíveis	O que o agente pode fazer.	Mover para: norte, sul, este, oeste.
Função de transição	Como cada ação altera o estado.	"Mover norte" $\rightarrow (x, y+1)$.
Objetivo (meta)	O estado que o agente quer alcançar.	Robô com entrega na posição (5,4).
Custo do caminho	Medida de eficiência (tempo, energia, distância).	Menor número de movimentos ou consumo de energia.

7

7

Etapas da formulação de problemas

1. Identificar o objetivo — o que o agente precisa alcançar.
2. Modelar o ambiente — quais são os estados e as restrições.
3. Definir ações válidas — o que o agente pode fazer.
4. Descrever a função de transição — o que acontece após cada ação.
5. Determinar o custo (opcional) — para encontrar soluções mais eficientes.

8

8

Etapas do processo de resolução de problemas

1. Formulação do problema → definir estados, ações e objetivos.
2. Pesquisa de soluções → explorar o espaço de estados (com algoritmos).
3. Seleção da melhor solução → otimizar tempo, custo ou distância.
4. Execução e aprendizagem → aplicar e ajustar a solução com base na experiência.

9

9

Diferença entre formulação e resolução

Etapa	Descrição
Formulação	Definir o problema (estados, ações, metas, custos).
Resolução	Aplicar algoritmos de procura para encontrar a solução.

Antes de um GPS “calcular a rota”, é preciso **formular o problema** (pontos de partida, chegada, estradas possíveis e custos de percurso).

10

10

Diferenças entre tipos de agentes

Tipo de agente	Como resolve problemas
Reativo	Responde a estímulos imediatos (não planeia).
Baseado em objetivos	Escolhe ações para atingir um objetivo.
Baseado em utilidade	Escolhe ações que maximizam uma medida de valor (melhor resultado possível).
Baseado em aprendizagem	Aprende com experiências passadas para resolver melhor no futuro.



SIMPLE REFLEX AGENTS



MODEL-BASED REFLEX AGENTS



GOAL-BASED AGENTS



UTILITY-BASED AGENTS



LEARNING AGENTS

11

11

Em resumo

- A formulação de problemas é o processo de definir o que o agente sabe, o que pode fazer e o que quer alcançar, transformando o mundo real num modelo computacional que permite à IA procurar soluções de forma inteligente.
- A resolução de problemas em IA é o processo pelo qual um agente inteligente planeia e executa ações para atingir um objetivo definido, usando busca, planeamento, heurísticas ou aprendizagem.

12

12

Exemplo: Problema de navegação

- Ir do ponto A ao ponto B.
- Estado inicial: posição atual.
- Ações: virar, seguir em frente, parar.
- Função de transição: cada ação move para uma nova posição.
- Objetivo: chegar ao destino da forma mais rápida e segura.
- Custo: Menor tempo de viagem.
- Técnicas usadas: algoritmos de procura.
- Aplicação real: GPS com cálculo de rotas em tempo real.

13

13

Exemplo: Problema do Puzzle-8

- “Puzzle-8” (tabuleiro 3x3 onde se move uma peça vazia para ordenar números).
- Estado inicial: disposição inicial das peças.
- Ações: mover peça para cima/baixo/esquerda/direita.
- Função de transição: Atualiza o tabuleiro com a nova posição da peça.
- Objetivo: chegar à disposição final ordenada.
- Custo: Número de movimentos necessários para resolver o puzzle.
- Técnicas usadas: procura em espaço de estados, heurísticas de distância.
- Aplicação real: otimização e planeamento de processos.

14

14

Exemplo: Problema dos jogos competitivos

- jogar xadrez ou damas (ou qualquer outro jogo com oponentes).
- Estado inicial: posição das peças no tabuleiro.
- Ações: jogadas possíveis.
- Função de transição: Atualiza o tabuleiro com a nova jogada.
- Objetivo: dar “xeque-mate” ao adversário.
- Custo: Número de movimentos necessários para ganhar o jogo.
- Técnicas usadas: algoritmos de procura adversária.
- Aplicação real: motores de xadrez como Stockfish ou AlphaZero.

15

15

Exemplo: Planeamento de tarefas

- um robô num armazém que tem de apanhar uma caixa e colocá-la numa prateleira.
- Estado inicial: robô sem caixa.
- Ações: mover-se, apanhar, largar.
- Função de transição: Cada ação modifica o estado do armazém.
- Objetivo: caixa colocada na prateleira correta.
- Custo: Tempo total ou energia consumida.
- Técnicas usadas: planeadores automáticos.
- Aplicação real: robôs logísticos da Amazon.

16

16

Problema de Procura

- Para executar alguma tarefa, um agente que planeia pode decidir o que fazer comparando diferentes sequências de ações possíveis.
 - para depois escolher a melhor sequência para executar
- Um **problema de procura** é o processo de procurar pela **melhor** sequência
- Resolver um problema de procura envolve:
formular problema → procurar solução → executar solução

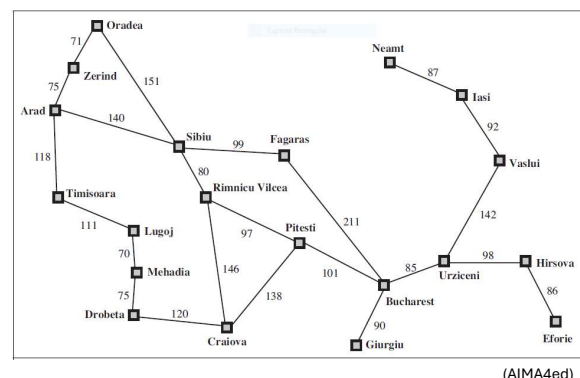


17

17

Exemplo: Roménia

- De férias na Roménia, atualmente em Arad, voo sai amanhã de Bucareste
- Formular problema:
 - espaço de estados: cidades
 - estado inicial: Arad
 - modelo de transições: permite mover entre cidades vizinhas
 - objetivo: Bucareste
 - custo = distância
- Solução: uma sequência de cidades
 - ex.: Arad → Sibiu → Fagaras → Bucareste



18

18

Solução versus Solução Ótima

- Uma **solução** para um problema de procura é uma sequência de ações que levam do estado inicial para o estado objetivo
- Uma **solução ótima** é uma solução com o menor custo de caminho dentre todas as possíveis
- Pode haver mais de uma solução ótima para um problema de procura

19

19

Espaço de Estados

- O conjunto de todos os estados acessíveis a partir de um estado inicial é chamado de **espaço de estados**
- Definido implicitamente pelos seguintes componentes: estado inicial, ações e modelo de transições
 - Os estados acessíveis são aqueles dados pelo modelo de transição.
- O espaço de estados forma um **grafo direcionado** em que os nós são estados e os arcos são ações

20

20

Selecionando um Espaço de Estados

- O mundo real é absurdamente complexo
 - o espaço de estados é uma **abstração**
- Estado (abstrato) = conjunto de estados reais
- Ação (abstrata) = combinação complexa de ações reais
 - ex., "Arad → Zerind" representa um conjunto complexo de rotas, desvios, paragens, etc.
 - Qualquer estado real do conjunto "Arad" deve levar a algum estado real "Zerind".
- Solução (abstrata) = conjunto de caminhos reais que são soluções no mundo real
- A abstração é útil se cada ação abstrata é mais fácil de executar que o problema original
- Consideração acerca da simplificação demasiada



21

21

Exemplo: Puzzle de 8 peças

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

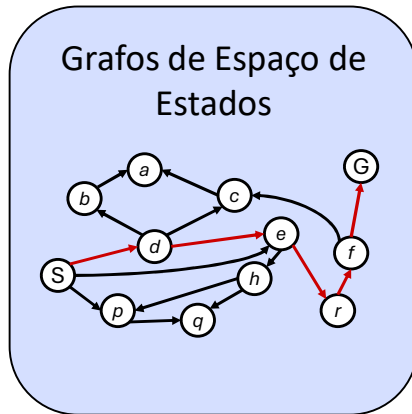
Goal State

- **Estados:** Especifica a posição de cada uma das peças e do espaço vazio
- **Estado inicial:** Qualquer um
- **Modelo de transição:** gera os estados válidos que resultam da tentativa de executar as quatro ações (mover espaço vazio para esquerda, direita, acima ou abaixo)
- **Teste de objetivo:** Verifica se o estado corrente corresponde à configuração objetivo.
- **Custo do caminho:** Cada passo custa 1, e assim o custo do caminho é o número de passos do caminho

22

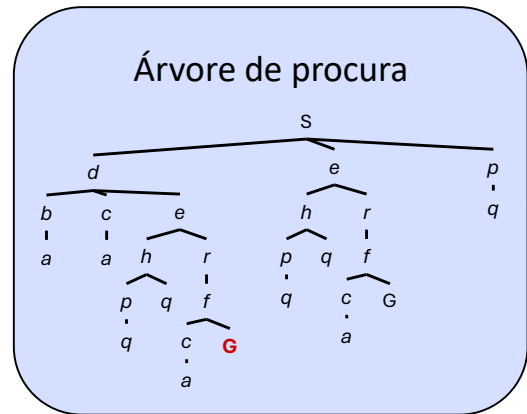
22

Grafos de Espaço de Estados vs. Árvores de procura



*Cada nó na árvore
de procura
corresponde a um
caminho no grafo
de espaço de
estados*

*Ambos são
construídos sob
pedido – tentamos
construir o mínimo
possível de cada um*

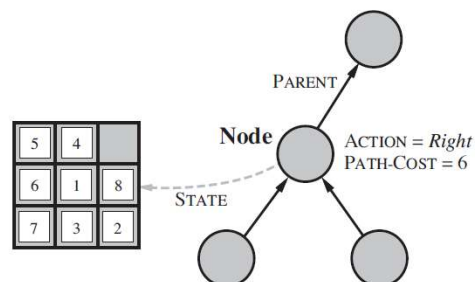


23

23

Estados vs. Nós

- Um **estado** é uma (representação de uma) configuração física
- Um **nó** é parte da árvore de procura e inclui **estado, nó pai, ação, custo do caminho**
 - Representado pela **estrutura de dados** denominada **Node**
 - Note que o campo PARENT permite organizar os nós em uma árvore

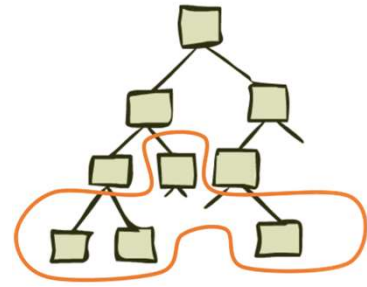


24

24

Procura em Árvore (Tree Search)

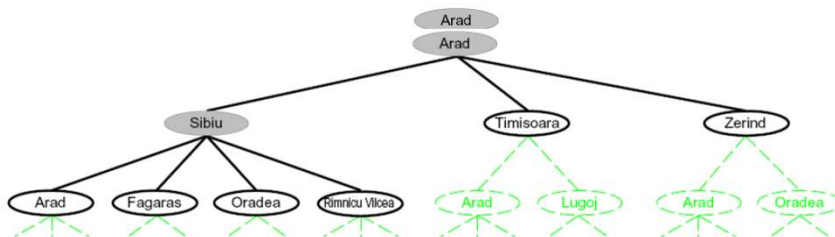
- Ideia: percorrer o espaço de estados com uma **árvore de procura**
 - raiz é o estado inicial
 - arestas são ações
 - nós (vértices) contêm elementos do espaço de estados
- Expandir o estado atual aplicando o modelo de transição para gerar novos estados
- O conjunto de nós folha disponíveis para expansão em um dado momento é denominado de **fronteira** ou **borda** (*frontier, fringe*)
- Procura: seguir um caminho, guardando os outros para tentar depois
- A **estratégia de procura** selecionada determina qual caminho seguir



25

25

Procura em Árvore

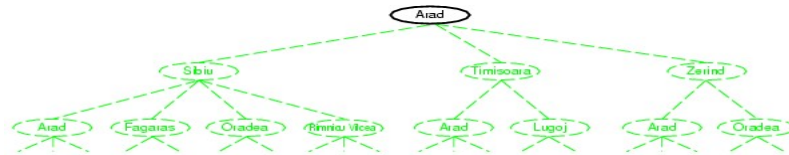


- Como a procura funciona:
 - Expande planos em potencial (nós da árvore)
 - Mantém uma **borda** (fronteira) de planos parciais sendo considerados;
 - Tenta expandir o mínimo de nós possível

26

26

Exemplo de árvore de procura



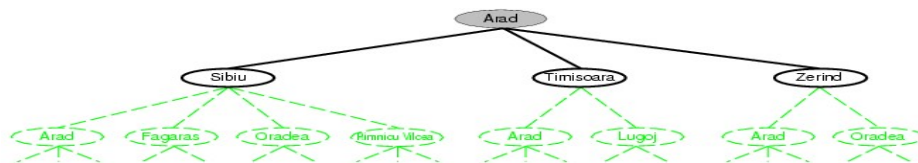
Estado inicial

Fronteira = {Arad}

27

27

Exemplo de árvore de procura



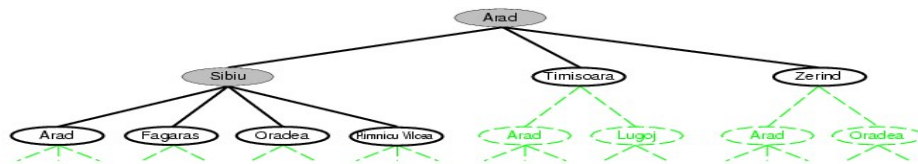
Depois de expandir Arad

Fronteira = {Sibiu, Timisoara, Zerind}

28

28

Exemplo de árvore de procura



Depois de expandir Sibiu

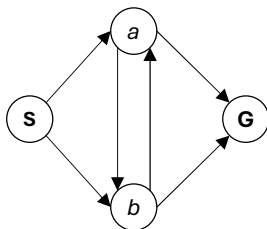
Fronteira = {Arad, Fagaras, Oradea, Himnicu Vilcea, Timisoara, Zerind}

29

29

Desafio: Grafos de Espaço de Estados vs. Árvores de procura

Considere este grafo com 4 estados:



O quão grande é a árvore de procura correspondente (a partir de **S**)?



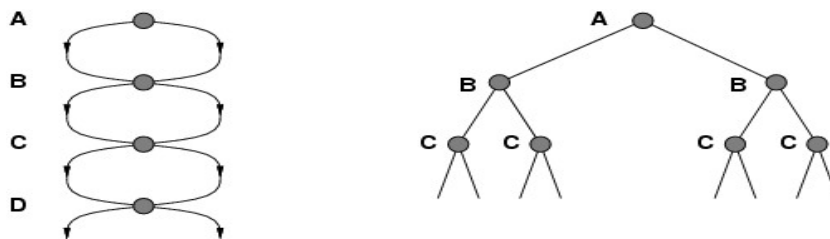
Em geral, há problemas que podem gerar **estados repetidos** em uma árvore de procura!

30

30

Estados repetidos

- A procura em árvore pode perder tempo expandindo nós já explorados antes.
- Estados repetidos podem
 - levar a loops infinitos
 - transformar um problema linear em um problema exponencial



31

31

Estados repetidos

- Comparar os nós prestes a serem expandidos com nós já visitados
- Se o nó já tiver sido visitado, será descartado; coleção “closed” armazena nós já visitados
- A procura percorre um grafo e não uma árvore

```

function GRAPH-SEARCH(problem, fringe) return a solution, or failure
    closed ← an empty set
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return node
        if STATE[node] is not in closed then
            add STATE[node] to closed
            for child-node in EXPAND(STATE[node], problem) do
                fringe ← INSERT(child-node, fringe)
            end
        end
    end
end
    
```

32

32

Tree Search vs Graph Search

- Em resumo:

- Há duas formas possíveis de implementar algoritmos de procura
- Graph Search é apenas uma extensão da Tree Search

Tree Search

```
open <- []
next <- start
while next isn't goal {
  open += successors of next
  next <- select from open
  remove next from open
}
return next
```

Graph Search

```
open <- []
closed <- []
next <- start
while next isn't goal {
  closed += next
  open += successors of next, which are not in closed
  next <- select from open
  remove next from open
}
return next
```

33

33

Propriedades de uma Estratégia de Procura

- Estratégias são avaliadas de acordo com os seguintes critérios:

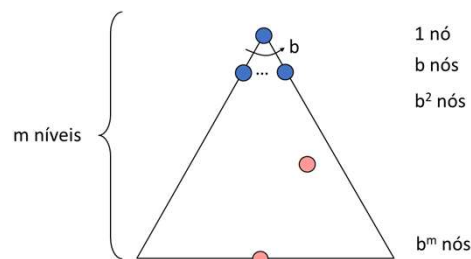
- **Completa?** Sempre encontra a solução, se alguma existe?
- **Complexidade de tempo:** Número de nós gerados no pior caso
- **Complexidade de espaço:** Número máximo de nós na memória no pior caso
- **Ótima?** Garante encontrar a solução ótima?

- Esboço de uma árvore de procura (pior caso):

- b é o máximo fator de ramificação
- m é a profundidade máxima
- pode haver soluções (em rosa) em vários níveis

- Número de nós na árvore?

- $1 + b + b^2 + \dots + b^m = O(b^{m+1})$



34

34

Estratégias de Procura Não Informada

- Estratégias de **procura não informada** (ou **procura cega** - *blind Search*) usam apenas a informação disponível na definição do problema para escolher o plano
 - Apenas geram sucessores e verificam se o estado objetivo foi atingido
- Estratégias de procura sem informação se distinguem pela **ordem** em que os nós são selecionados na borda para ser expandidos.
 1. Procura em Extensão (*breadth-first search, BFS*)
 2. Procura com Custo Uniforme (*uniform-cost search, UCS*)
 3. Procura em Profundidade (*depth-first search, DFS*)

35

35

Procura em Extensão (*Breadth-First Search* - BFS)

- Estratégia: expandir o nó não expandido que esteja mais perto da raiz
- “BFS: do all the shallow things before we go into the deep things”
- Todos nós que estão numa profundidade d da árvore serão expandidos e visitados antes que os nós que estão numa profundidade $d+1$
- **Implementação:**
 - a *borda* é uma fila FIFO (first-in, first-out), isto é, novos itens entram no final



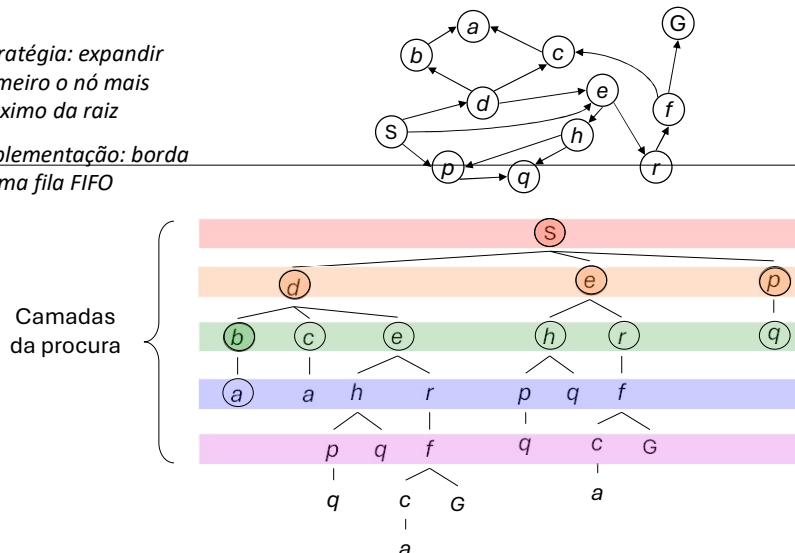
36

36

BFS: exemplo

Estratégia: expandir primeiro o nó mais próximo da raiz

Implementação: borda é uma fila FIFO



37

37

BFS: requisitos de tempo e memória

- Suposições
 - procura com fator de ramificação $b=10$
 - 1.000.000 nós podem ser gerados por segundo
 - Um nó exige 1KB de espaço

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

38

38

Procura com Custo Uniforme (Uniform Cost Search - UCS)

- Similar à BFS, só que expande o nó ainda não expandido n que tenha o custo de caminho $g(n)$ mais baixo
- **Implementação:**
 - *fronteira* = fila ordenada por $g(n)$ (i.e., uma **fila de prioridades**)
- Equivalente à procura em extensão se os custos são todos iguais
- Particularidades da UCS:
 - O teste de objetivo é aplicado a um nó quando ele é selecionado para expansão, e não quando ele é gerado
 - Há um teste para verificar se um caminho para um nó na fronteira é mais curto do que algum previamente encontrado



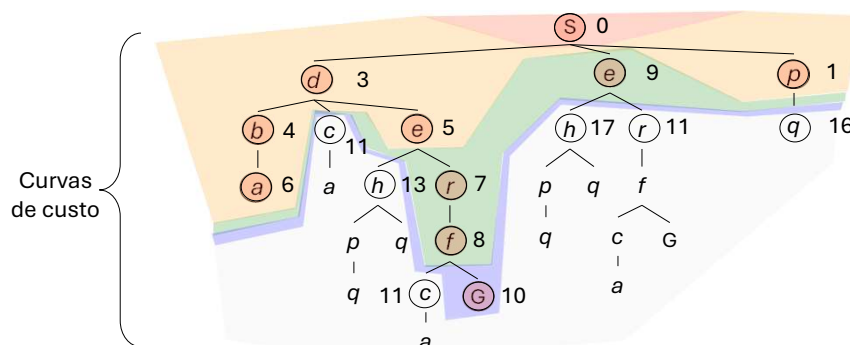
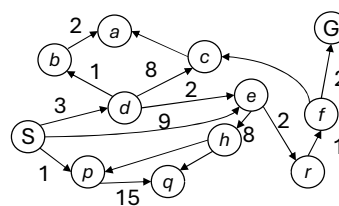
39

39

UCS: Exemplo

Estratégia: expandir o nó mais barato primeiro:

Borda é uma fila de prioridade com chave igual ao custo cumulativo, $g(n)$



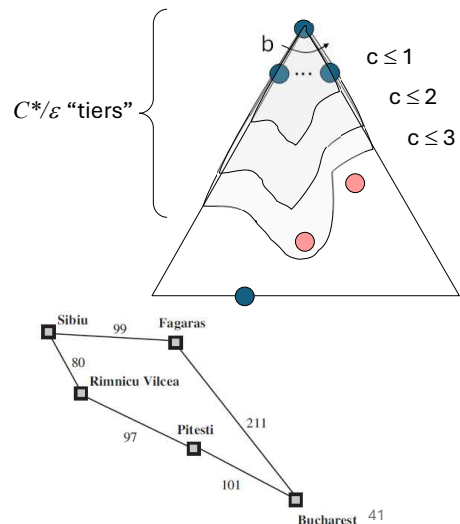
40

40

UCS: algoritmo e exemplo

```

function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by PATH-COST, with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier) /* chooses the lowest-cost node in frontier */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
    
```



41

Procura em Profundidade (Depth First Search - DFS)

Ordem de expansão dos nós:

- Sempre expande o nó no nível mais profundo da árvore:
 - Nó raiz
 - Primeiro nó de profundidade 1
 - Primeiro nó de profundidade 2, etc. de profundidade 2, etc.
- Quando um nó final não final não é solução, o algoritmo volta para expandir os nós que ainda estão na fronteira do espaço de estados
- Algoritmo: Algoritmo:


```

função Procura-em-Profundidade (problema)
  retorna uma solução ou falha
  Procura-Genérica (problema, Inserir-no-Começo)
      
```
- O nó de maior profundidade mais a esquerda é escolhido para gerar sucessores
- Quando é expandido um nó de maior profundidade, a procura chega a um nó sem sucessor, logo o algoritmo expande o próximo nó com menor profundidade



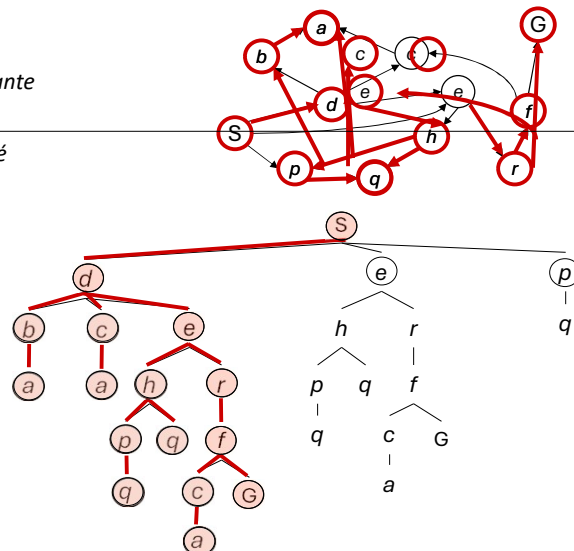
42

42

DFS: exemplo

Estratégia: expandir primeiro o nó mais distante da raiz

Implementação: borda é uma pilha LIFO (Stack)



43

43

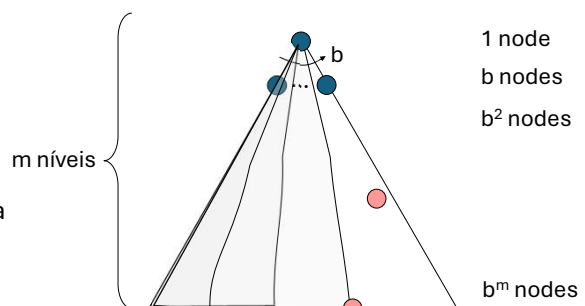
DFS

Vantagem:

- Requer pouca memória
 - O nó objetivo pode vir a ser encontrado sem examinar a árvore por completo

Desvantagem:

- É importante que cada sequência possível possa terminar
 - Se não o algoritmo “desce” indefinidamente



44

44